

Taller 1: Implementación de un Intérprete de Comandos Simple en Linux

Integrantes:

- Felipe Zuñiga
- Lukas Alvarado
- Gustavo Fernandez

Profesor: Juan Felipe González Saavedra

Fecha de entrega: 20 de mayo de 2026

Repositorio del proyecto: <https://github.com/Lukaaas12/Taller1SistemasOperativos.git>

Índice

1. Resumen
2. Introducción
3. Arquitectura General de la Solución
4. Implementación del Prompt
5. Lectura y Parseo de Comandos
6. Comando exit
7. Manejo de Errores
8. Ejecución Concurrente
9. Conclusiones
10. Referencias

Resumen

El presente informe documenta el diseño e implementación de un intérprete de comandos (shell) simple en Linux, desarrollado en lenguaje C para la asignatura Sistemas Operativos. El proyecto tuvo como objetivo comprender el manejo de procesos concurrentes mediante las llamadas al sistema `fork()`, `execvp()` y `waitpid()`, además del uso de `getline()` para la lectura dinámica de comandos. La shell implementada permite ejecutar comandos en foreground, soporta múltiples argumentos, detecta comandos inexistentes y finaliza correctamente mediante el comando `exit`. El informe se estructura bajo formato académico APA Séptima Edición e incorpora análisis técnico, evidencias de funcionamiento y conclusiones relacionadas con el aprendizaje obtenido durante el desarrollo del taller.

Introducción

En los sistemas operativos tipo Unix/Linux, la shell constituye el principal mecanismo de interacción entre el usuario y el sistema. Su función consiste en interpretar instrucciones ingresadas desde la terminal y ejecutar los programas correspondientes mediante la creación y administración de procesos.

El presente taller tiene como finalidad aplicar conceptos fundamentales de concurrencia y administración de procesos en Linux, utilizando llamadas al sistema proporcionadas por POSIX. A través del desarrollo de una shell simple se logra comprender el ciclo de vida de un proceso: creación mediante `fork()`, sustitución de imagen con `execvp()` y sincronización mediante `waitpid()`.

Además de la implementación práctica, el proyecto permite analizar cómo las funciones de biblioteca de C interactúan con el kernel a través de syscalls como `read()`, `write()` y `_exit()`. Este informe describe la arquitectura del programa, el funcionamiento de cada requisito solicitado en la rúbrica oficial y las evidencias experimentales obtenidas durante la ejecución del software.

Arquitectura General de la Solución

La shell fue desarrollada en lenguaje C bajo el estándar C99/C11 utilizando GCC como compilador. El programa implementa el modelo clásico REPL (Read-Eval-Print Loop), cuyo flujo consiste en:

1. Mostrar el prompt.
2. Leer la entrada del usuario.

3. Parsear el comando y sus argumentos.
4. Crear un proceso hijo mediante `fork()`.
5. Ejecutar el comando usando `execvp()`.
6. Esperar la finalización con `waitpid()`.
7. Repetir el ciclo hasta recibir el comando `exit`.

La solución se implementó en un único archivo fuente denominado `mi_shell.c` y fue compilada utilizando:

```
gcc -Wall -Wextra -o mi_shell mi_shell.c
```

Implementación del Prompt

El prompt permite identificar que la shell se encuentra esperando instrucciones. Para ello se utiliza `printf()` seguido de `fflush(stdout)`, asegurando que el buffer de salida sea vaciado inmediatamente antes de bloquear la ejecución con `getline()`.

Aunque `printf()` pertenece a la biblioteca estándar de C, internamente utiliza la llamada al sistema `write()` sobre `stdout`.

```
mi_shell>  
mi_shell>  
mi_shell>  
mi_shell>  
mi_shell>  
mi_shell>  
mi_shell> 
```

Ilustración 1: Comportamiento del ciclo REPL ante la lectura de entradas vacías

Lectura y Parseo de Comandos

La lectura de comandos se realiza mediante `getline()`, función que administra dinámicamente el buffer de entrada y evita truncamientos. Posteriormente, `strtok()` separa la línea ingresada utilizando espacios y saltos de línea como delimitadores.

Los argumentos se almacenan en un arreglo dinámico de punteros `char**`, el cual puede expandirse mediante `realloc()` para soportar una cantidad indeterminada de parámetros,

cumpliendo completamente el requerimiento de la rúbrica.

```
lukitasalvarado2@cloudshell:~$ gcc -Wall -Wextra -o mi_shell mi_shell.c
lukitasalvarado2@cloudshell:~$ ./mi_shell
mi_shell> ls -la /tmp
total 76
drwxrwxrwt 1 root      root      4096 May 20 00:39 .
drwxr-xr-x 1 root      root      4096 May 20 00:28 ..
drwx----- 2 lukitasalvarado2 lukitasalvarado2 4096 May 20 00:28 cloudcode-temp4A4TJs
drwx----- 2 lukitasalvarado2 lukitasalvarado2 4096 May 20 00:28 cloudcode-tempJ6AjLX
drwx----- 2 lukitasalvarado2 lukitasalvarado2 4096 May 20 00:28 cloudcode-tempQ2rifi
drwx----- 2 lukitasalvarado2 lukitasalvarado2 4096 May 20 00:28 cloudcode-tempvkHHme
-rw-r--r-- 1 lukitasalvarado2 lukitasalvarado2   77 May 20 00:28 code-oss-vm-session-stat
drwxr-xr-x 3 lukitasalvarado2 lukitasalvarado2 4096 May 20 00:28 gemini
-rw-r--r-- 1 lukitasalvarado2 lukitasalvarado2 4306 May 20 00:28 minikube_delete_2ead6aae
drwxrwxr-x 3 lukitasalvarado2 lukitasalvarado2 4096 May 20 00:28 node-compile-cache
-rw-rw-r-- 1 lukitasalvarado2 lukitasalvarado2    0 May 20 00:28 restore_bashrc.lock
-rw-rw-r-- 1 lukitasalvarado2 lukitasalvarado2    0 May 20 00:28 restore_profile.lock
-rw----- 1 root      root        0 May 20 00:28 tmp.4tr0hpjKRt
drwx----- 4 lukitasalvarado2 lukitasalvarado2 4096 May 20 00:37 tmp.NugP5XvTXu
drwx----- 4 lukitasalvarado2 lukitasalvarado2 4096 May 20 00:32 tmp.olS9B1x55p
drwx----- 4 lukitasalvarado2 lukitasalvarado2 4096 May 20 00:37 tmp.Qwgz4sSgE3
drwx----- 4 lukitasalvarado2 lukitasalvarado2 4096 May 20 00:28 tmp.up2W3pOmJ2
drwx----- 4 lukitasalvarado2 lukitasalvarado2 4096 May 20 00:36 tmp.XynrJUoZHp
drwxr-xr-x 4 lukitasalvarado2 lukitasalvarado2 4096 May 20 00:28 tmp.YDJTtSmk7K
drwx----- 2 lukitasalvarado2 lukitasalvarado2 4096 May 20 00:28 tmux-1000
srwxr-xr-x 1 lukitasalvarado2 lukitasalvarado2    0 May 20 00:28 vscode-git-7b7992ce83.s
srwxr-xr-x 1 lukitasalvarado2 lukitasalvarado2    0 May 20 00:28 vscode-ipc-a4722ela-4b8e
drwxr-xr-x 2 lukitasalvarado2 lukitasalvarado2 4096 May 20 00:28 vscode-skaffold-events-1
mi_shell> █
```

Ilustración 2: Evidencia de compilación y ejecución concurrente con múltiples argumentos en foreground

Comando exit

El comando exit se detecta utilizando strcmp(). Cuando el usuario lo ingresa, el programa libera la memoria utilizada y finaliza el ciclo principal. Posteriormente, el proceso termina limpiamente mediante exit(), el cual internamente invoca la syscall _exit().

```
mi_shell> exit
lukitasalvarado2@cloudshell:~$ █
```

Ilustración 3: Finalización de la Shell mediante el comando exit.

Manejo de Errores

Cuando el usuario ingresa un comando inexistente, execvp() retorna -1 y establece el valor correspondiente en errno. El programa utiliza perror() para mostrar un mensaje descriptivo sin finalizar la shell principal, permitiendo continuar la ejecución normalmente.

```
mi_shell> comando_falso_test
Error en mi_shell: No such file or directory
mi_shell> █
```

Ilustración 4: Robustez y manejo de errores continuables ante comandos inexistentes.

Ejecución Concurrente

La ejecución concurrente constituye el núcleo funcional de la shell. Mediante `fork()` se crea un proceso hijo encargado de ejecutar el comando solicitado. Luego, `execvp()` reemplaza completamente la imagen del proceso hijo por el programa correspondiente.

Mientras tanto, el proceso padre utiliza `waitpid()` para esperar la finalización del hijo, implementando correctamente la ejecución en foreground exigida por la rúbrica.

Conclusiones

El desarrollo de este proyecto permitió comprender de manera práctica el funcionamiento interno de los procesos en Linux y la interacción entre el espacio de usuario y el kernel. La implementación de una shell simple evidenció la importancia de las llamadas al sistema `fork()`, `execvp()` y `waitpid()` como mecanismos fundamentales para la concurrencia en sistemas Unix.

Además, el trabajo fortaleció habilidades relacionadas con manejo dinámico de memoria, tratamiento de errores y administración de procesos hijos. Desde el punto de vista académico, el proyecto cumple con todos los requisitos solicitados en la rúbrica oficial y presenta evidencias claras del correcto funcionamiento de la solución implementada.

El informe fue reorganizado y estructurado siguiendo criterios formales de presentación académica y normas APA Séptima Edición, mejorando significativamente la claridad, orden y profesionalismo del documento original.

Referencias

Kerrisk, M. (2010). The Linux programming interface: A Linux and UNIX system programming handbook. No Starch Press.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). Wiley.

Stevens, W. R., & Rago, S. A. (2013). Advanced programming in the UNIX environment (3rd ed.). Addison-Wesley.

The Open Group. (2018). The Open Group Base Specifications Issue 7 — fork(2).
<https://pubs.opengroup.org/>